

The Name of the Title Is Hope

Anonymous Author(s)

Abstract

Automated unit test generation is essential for improving software reliability, yet existing techniques struggle to handle complex program semantics, especially when path reachability depends on implicit object states and inter-procedural interactions. Recent large language model (LLM)-based approaches can directly generate executable tests, but often fail to satisfy hidden preconditions that are not locally visible, resulting in low coverage. In this paper, we propose COGPATH, an LLM-driven framework that improves test generation by making implicit constraints explicit and reducing irrelevant context. Specifically, we introduce *Constraints-Hints* to expose path conditions as structured input requirements, and a coverage-driven backward slicing technique to retain only statements relevant to target branches. These components are integrated into an iterative pipeline that combines generation, execution, and repair. Experimental results on real-world Java projects show that COGPATH consistently achieves higher branch coverage, particularly for paths requiring precise object state construction. [xtz:need refine]

Keywords

Automated Unit Test Generation, Large Language Models, Software Testing, Program Analysis, Branch Coverage, Constraint-Guided Testing, Program Slicing

ACM Reference Format:

Anonymous Author(s). 2018. The Name of the Title Is Hope. *J. ACM* 37, 4, Article 111 (August 2018), ?? pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Automated unit test generation plays a critical role in improving software reliability and reducing developer effort. Traditional techniques, such as symbolic execution and search-based software testing (SBST), have made substantial progress in systematically exploring input spaces and maximizing structural coverage [???]. However, these approaches often face fundamental limitations in scalability and in handling complex program semantics. In particular, they struggle to capture implicit preconditions that arise from intricate object states and inter-procedural interactions, where the conditions required to reach a target path are established outside the local method scope through sequences of prior method calls [?].

Recent advances demonstrate that large language models (LLMs) can directly synthesize executable unit tests, significantly alleviating manual effort [?]. To handle complex methods and improve

coverage, subsequent research mainly pursues two directions. The first is *feedback-driven refinement*: CoverUp [?] iteratively targets untested regions using dynamic coverage analysis, while YATE [?] repairs failing tests to enhance validity. The second direction emphasizes *structural decomposition and path sensitivity*. To simplify complex logic, HITS [?] prompts LLMs to decompose methods into fragments. Advancing this using program analysis, SymPrompt [?] extracts branch constraints from Control Flow Graphs (CFGs), whereas Panta [?] fuses static CFG analysis with dynamic coverage feedback to identify uncovered paths. Building upon this trajectory, JUnitGenie [?] achieves strict path sensitivity by isolating the minimal, path-specific context required for targeted generation.

Despite these promising results, a substantial coverage gap still remains, especially on complex branches whose reachability depends on non-local object states and long dependency chains. This suggests that the remaining challenge is not simply a lack of feedback or prompting sophistication, but a deeper mismatch between the information exposed to the LLM and the information actually required to cover a target branch. ① One important dimension of this mismatch is *what to satisfy*. In many cases, branch reachability depends on implicit preconditions scattered across long dependency chains, requiring the model to infer hidden constraints through multiple variable assignments and state transitions before it can synthesize a valid test. Existing approaches typically provide only the target branch structure and compress constraint inference and test synthesis into a single generation step, without any explicit intermediate output or independent verification of the inferred constraints. As a result, the LLM can easily skip or compress critical reasoning steps, producing tests that appear plausible but fail to satisfy the actual conditions required to reach the target branch. This problem becomes particularly severe when the underlying dependency chain is long or semantically complex. ② The second dimension concerns *what to attend to*. Existing methods usually expose the entire method body to the LLM, even though only a small subset of statements has real data or control dependence on the target branch. The resulting context is therefore structurally misaligned with the branch-coverage objective: coverage-critical dependencies are diluted by irrelevant code, attention is spread across non-essential statements, and the model must reason under a representation that over-emphasizes what is visible rather than what is truly necessary. This not only amplifies cognitive burden, but also makes precise state construction substantially less likely. Even without explicit distractors, LLM performance can already degrade as context length grows [?]; in branch-oriented test generation, this effect is further magnified because the relevant context is often deeply buried inside much longer method bodies. Taken together, these two issues suggest a systematic form of *cognitive misalignment* in LLM-based unit test generation: the information shown to the model is often misaligned with the information actually needed to cover a target branch. This misalignment has two complementary dimensions, namely *what to satisfy* and *what to attend to*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

To address this cognitive misalignment, we propose COGPATH, an LLM-driven framework for automated unit test generation that explicitly tackles both dimensions of the gap, by making critical constraints explicit and concentrating the model's attention on branch-relevant program context. First, *Constraint-Hints* addresses the *what to satisfy* dimension by making the execution conditions of uncovered paths explicit. Instead of forcing the LLM to deduce implicit constraints from raw code, this mechanism extracts path conditions and translates them into actionable guidance. It drastically reduces the model's reasoning difficulty by directly suggesting *how* to satisfy a target condition—such as invoking specific setter methods to establish the required object state. By surfacing these hidden cross-method preconditions, it actively guides the LLM to construct valid initialization sequences, ensuring complex dependencies are properly addressed rather than silently skipped. Second, *coverage-driven backward slicing* addresses the *what to attend to* dimension. Rather than supplying the full method body, the slicer isolates the minimal set of statements that genuinely influence an uncovered branch, including cross-method dependencies such as field initializations and setter methods. This restructures the prompt context around branch-relevant dependencies, preventing the model's attention from being diffused across irrelevant code. The slicing is triggered selectively when coverage plateaus, targeting branches that resist standard generation. Both components are integrated into an iterative loop of generation, execution, and repair. Coverage feedback drives target selection each round, progressively exposing branches where the current context remains misaligned with their reachability requirements, until coverage converges or the budget is exhausted.

Evaluation. We evaluate COGPATH on 14 Java projects from Defects4J v2.0.1, covering 130 classes and 2971 methods with cyclomatic complexity greater than 10. Baselines include SymPrompt, HITS, and Panta, representing complementary design philosophies. COGPATH achieves 44.41% average branch coverage, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Gains are most pronounced on logic-heavy subjects with rich cross-method dependencies: on Gson, COGPATH reaches 41.24% versus SymPrompt's 3.59%; on Jsoup, it improves over HITS by 36.4%.

Ablation study confirms the complementary roles of both components. Removing *Backward Slicing* drops coverage from 44.41% to 38.47%, with the largest degradation on context-heavy subjects (e.g., Csv: 52.29%→24.80%). Removing *Constraint-Hints* drops coverage to 39.79%, most severely on subjects with object-state dependencies (e.g., JacksonDatabind: 17.85%→9.33%). Finally, COGPATH generalizes across backbone LLMs, achieving 44.41% (QC3-30B), 83.47% (QW3.5-397B), and 80.12% (DS-V3.1), confirming that its benefit derives from restructuring branch-relevant information rather than relying on any specific model.

In summary, this paper makes the following contributions:

- We identify a key limitation of existing LLM-based test generation approaches in handling implicit, cross-method constraints.
- We propose *Constraints-Hints*, a mechanism that makes path conditions explicit and guides LLM reasoning.
- We introduce a coverage-driven backward slicing technique to eliminate irrelevant context and expose critical dependencies.

- We design an iterative framework that integrates generation, execution feedback, and repair for sustained coverage improvement.
- We empirically demonstrate the effectiveness of our approach on real-world software systems.

2 MOTIVATION

Recent LLM-based unit test generation approaches, such as SymPrompt [?], HITS [?], Panta [?], and JunitGenie [?], have achieved remarkable progress in automating test generation for complex methods, significantly reducing the manual effort required from developers. However, when the reachability of a target branch depends on *object states established by prior method calls*, existing approaches still struggle to guide LLMs in constructing the appropriate external context, even when augmented with CFG-based path analysis.

The fundamental limitation is that these approaches typically supply the *entire method body* as the LLM's input context. In real-world projects, however, the conditions necessary to reach a target branch are often not visible within the method body itself—they are implicit in a sequence of initialization and state-setting operations that must occur *before* the method is invoked [?]. The irrelevant statements in the method body not only consume the context window but also obscure these cross-method dependencies, leaving the LLM unable to infer the required preconditions.

A Motivating Example. [xtz:lack example fot backward slicing] Consider the `addValue()` method from Apache Commons Math [?], shown in Listing ??.

Listing 1: Target branch example from Apache Commons Math

```
1 // org.apache.commons.math3.stat.descriptive.  
2   SummaryStatistics  
3 public void addValue(double value) {  
4     sumImpl.increment(value);  
5     ...  
6     if (meanImpl != mean) { // target branch  
7         meanImpl.increment(value);  
8     }  
9     ...  
10    n++;  
11 }
```

Suppose a test generation tool identifies that the branch at line 9 (`meanImpl != mean`) has not been covered and attempts to generate a test to reach it. Existing approaches typically provide the entire method body alongside CFG path information to the LLM. However, the LLM faces a critical inferential barrier: the condition `meanImpl != mean` can only be satisfied if `setMeanImpl()` was called *prior* to `addValue()`—a dependency that is entirely absent from the method body. Consequently, the LLM tends to generate a test such as:

Listing 2: Typical LLM-generated test (target branch unreachable)

```
1 // meanImpl == mean always holds; target branch is  
2   never triggered  
3 SummaryStatistics stats = new SummaryStatistics();
```

```
stats.addValue(1.0);
```

This test compiles and executes without error, yet the target branch is never triggered. The correct test requires establishing the necessary precondition *before* invoking `addValue()`:

Listing 3: Correct test (target branch reachable)

```
// Establish precondition via setMeanImpl() before
// calling addValue()
SummaryStatistics stats = new SummaryStatistics();
stats.setMeanImpl(new CustomMeanImpl()); // makes
// meanImpl != mean
stats.addValue(1.0); // target
// branch now reachable
```

This example illustrates two core challenges that existing approaches fail to address: (1) the method body contains numerous statements irrelevant to the target branch, which dilute the LLM’s attention and impede precise reasoning; and (2) the true precondition for reaching the target branch is a cross-method dependency invisible within the method body itself.

todo: bs example

Our Approach. To address these challenges, we propose two complementary mechanisms that reduce LLM reasoning complexity from the perspectives of *information reduction* and *condition explicitness*:

- **Backward Slicing.** Starting from the target branch condition `meanImpl != mean`, we perform backward slicing to trace all statements with data or control dependencies on `meanImpl` and `mean`, including field declarations, default assignments in the constructor, and the definition of `setMeanImpl()`. This eliminates six irrelevant increment calls and other unrelated branches from the context, allowing the LLM to focus on statements that govern branch reachability.
- **Constraint Hints.** Building on the slice, we explicitly provide the LLM with the path condition and how it can be established, e.g., “To trigger the target branch, `meanImpl != mean` must hold; this can be achieved by calling `setMeanImpl(customImpl)` before `addValue()`.” Rather than requiring the LLM to infer implicit cross-method dependencies from source code, this mechanism directly guides the LLM toward constructing the correct initialization sequence.

Together, these two mechanisms systematically reduce the difficulty of reasoning about complex branch conditions, enabling LLMs to generate tests that reach previously uncovered paths.

3 Methodology

3.1 Overview

COGPATH is a coverage-guided iterative framework for automated unit test generation, illustrated in Figure ?? . Given a method under test and an initial test suite, COGPATH builds the CFG, enumerates feasible paths, collects dynamic coverage feedback to identify uncovered branches, and then enters an iterative loop to systematically handle these branches. Each iteration proceeds in three steps. First, the framework selects an uncovered path and applies *Constraints-Hints* to extract explicit branch conditions as structured hints for the generation prompt. Second, if coverage has stagnated,

Backward Slicing replaces the full method body with the minimal set of branch-relevant statements. Third, the LLM generates candidate tests, which are compiled and executed; passing tests are merged into the suite, and failing tests enter a repair phase. Coverage is then re-measured to guide the next iteration. The loop terminates when target coverage is reached, the iteration budget is exhausted, or coverage fails to improve for a configurable number of consecutive iterations.

In the following sections, we will provide detailed introductions about COGPATH.

COGPATH is a coverage-guided iterative framework for automated unit test generation. The overall workflow is illustrated in Figure ?? . Starting from the method under test (MUT) and an initial test suite (possibly empty), the framework performs static analysis to construct the control-flow graph and enumerate feasible execution paths. Dynamic execution feedback is then collected to identify uncovered lines and branches.

To efficiently exercise uncovered paths, COGPATH integrates two complementary mechanisms. First, a *Constraints-Hints* module derives explicit branch conditions along a selected path and exposes them as structured hints in the test-generation prompt, making hidden input requirements visible to the LLM. Second, a *Backward Slicing* module is triggered when coverage improvement stagnates, extracting the minimal set of statements that influence a stubborn uncovered path and supplying them as targeted context.

Each iteration proceeds as follows. The framework selects uncovered paths, optionally derives constraint hints, and invokes the test-generation LLM to produce candidate tests. These candidates are compiled and executed to validate their correctness. Failing tests are optionally forwarded to a repair phase. Successfully validated tests are incorporated into the test suite and coverage is re-measured, closing the feedback loop. The process terminates when the target coverage is reached, the iteration budget is exhausted, or coverage ceases to improve for a configurable number of iterations.

3.2 Constraints-Hints for Path Solving

Constraints-Hints addresses the implicit precondition problem by decoupling constraint inference from test synthesis. Rather than asking the LLM to simultaneously reason about reachability and generate code, we introduce a dedicated constraint-solving step that makes branch preconditions explicit before generation begins. For methods with complex branch logic, directly instructing an LLM to cover a specific execution path often fails because branch predicates are buried within nested conditionals, method calls, or arithmetic expressions. Consequently, the model cannot reliably infer the precise input conditions required to reach the desired path. To address this limitation, we introduce *Constraints-Hints*, a mechanism that translates implicit branch conditions into explicit and structured input requirements.

Path formulation. Let $P = \langle b_1, b_2, \dots, b_k \rangle$ denote the sequence of branch decisions along a target execution path π , where each predicate $b_i(\mathbf{x}) \in \{\text{true}, \text{false}\}$ is evaluated over the program input vector $\mathbf{x}$. Covering the path requires an input assignment $\mathbf{x}^*$ that satisfies all branch predicates along the path:

$$\mathbf{x}^* \in \mathcal{F}(\pi) = \{\mathbf{x} \mid b_1(\mathbf{x}) \wedge b_2(\mathbf{x}) \wedge \dots \wedge b_k(\mathbf{x})\}. \quad (1)$$